



Penetration Testing

PoolParty (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	29
Our Methodology	30
Disclaimers	32
About Hashlock	33

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The PoolParty team partnered with Hashlock to conduct a security Penetration Test of their PoolParty Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

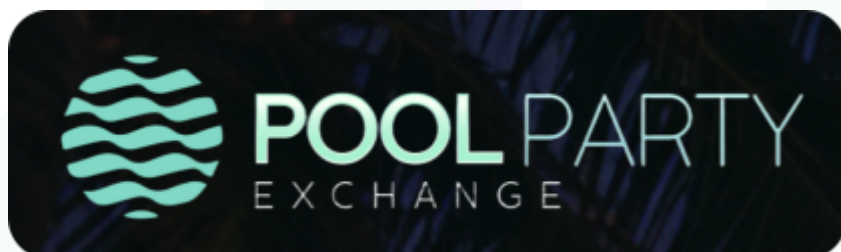
Project Context

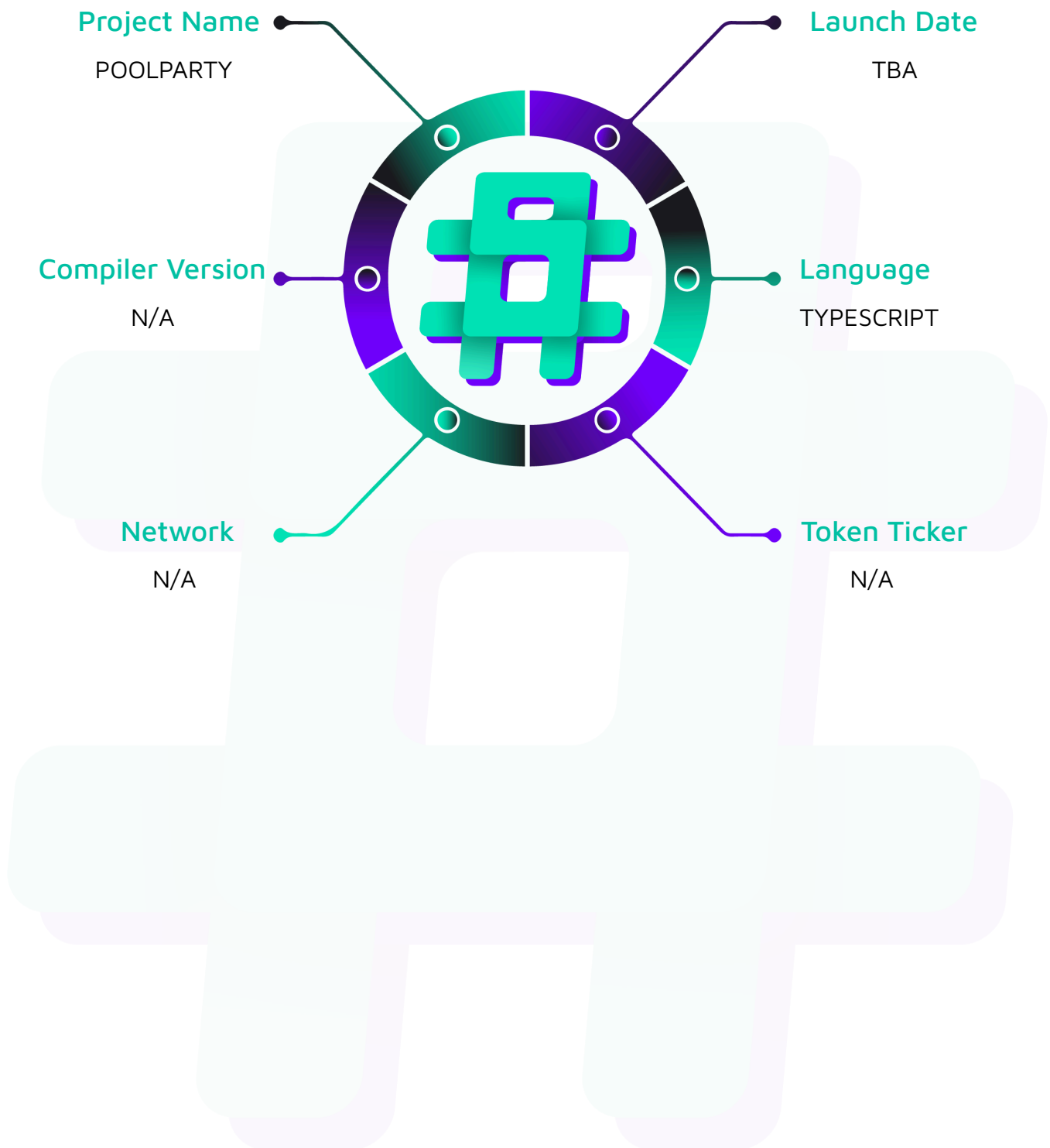
PoolParty™ is a decentralized trading platform combining DEX and OTC features. It allows users to list and trade tokens peer-to-peer with no price impact or fees. Buyers can access discounted tokens, while sellers earn ETH rewards. Users can also stake \$SPLASH to earn passive income.

Project Name: PoolParty

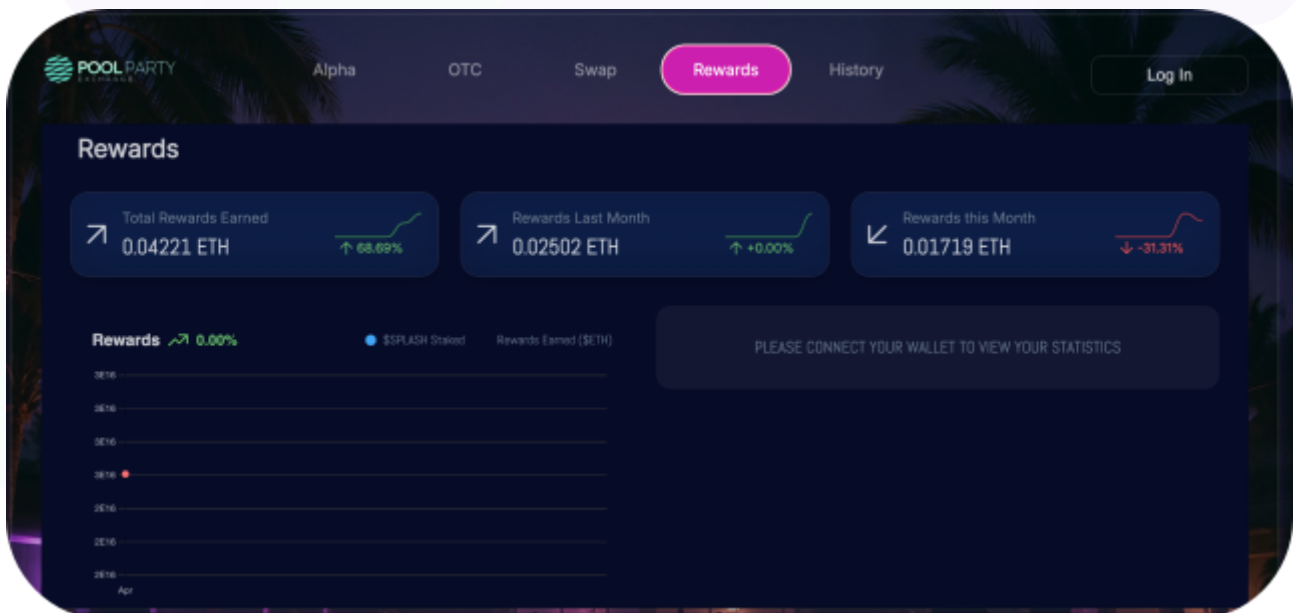
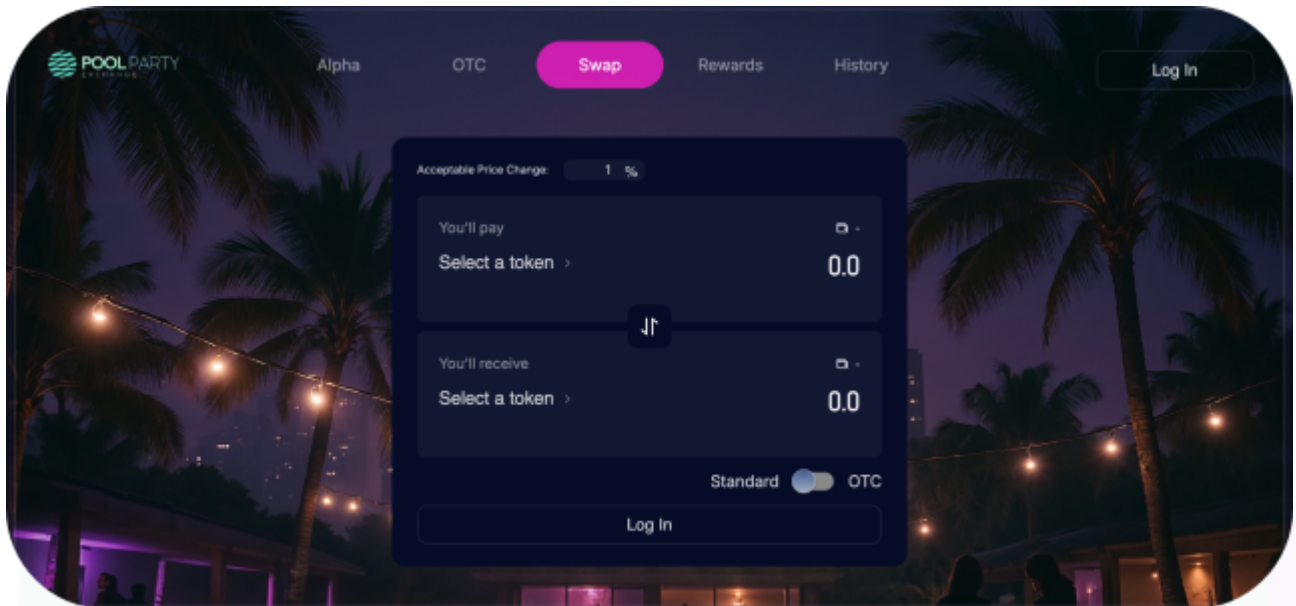
Project Type: dApp

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the PoolParty project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	PoolParty Project code base
Platform	Web apps & APIs
Penetration Test Date	June, 2025
Repository/Website URL/IP Tested	https://github.com/PoolParty-Exchange/PoolParty_App_Development

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed, and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High-severity vulnerability

4 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

2 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] AccountDetails.js - Insecure file upload

Description

The application accepts file uploads and directly converts them to base64 without any validation on file type, size, or content.

Vulnerability Details

In AccountDetails.js:

```
<FileBase64 multiple={false} onDone={v => setNewProfilePhoto(v?.base64?.toString())} />
```

Impact

This allows for uploading of malicious files like SVGs with embedded JavaScript (XSS), extremely large files causing DoS, or inappropriate content.

Recommendation

Implement proper file validation, including MIME type verification, size limits, and content scanning, before accepting uploads. The image file should have enforced target extensions on saving the file.

Status

Resolved

Medium

[M-01] Multiple endpoints - Double parsing of JSON in API endpoints can cause request processing errors

Description

The `postreferralswap` endpoint unnecessarily parses and re-stringsifies JSON data, potentially causing data corruption or errors.

Vulnerability Details

In `postreferralswap.js`:

```
let verifyBody = JSON.parse(req.body);  
  
// ...later...  
  
body: JSON.stringify(verifyBody),
```

Impact

This can cause errors since the body is already parsed by the framework, leading to unexpected behavior when processing requests. Worst case, these requests will keep on failing, making these functionalities not work.

Recommendation

Remove redundant double JSON parsing and directly use the parsed body object.

Status

Resolved

[M-02] Multiple endpoints - Lack of fetch error/timeout handling

Description

API calls to external services have no timeout configurations or retry logic, leading to potential resource exhaustion.

Vulnerability Details

Throughout the codebase, fetch operations are performed without timeouts like example below:

```
const response = await fetch('/api/profile/linktelegram', { /* no timeout */ })
```

Impact

Under degraded network conditions or unavailability of the external APIs, requests can hang indefinitely, exhausting connection pools and server resources.

Recommendation

Add timeout configurations to all fetch operations and implement retry logic with exponential backoff for transient failures.

Status

Resolved

[M-03] Multiple endpoints - Second-order DoS via external API

Description

The application makes calls to external APIs without implementing proper rate limiting on its side, which can lead to unexpected failures if the external APIs have stricter rate limits. However, it is not possible to confirm that issue fully without testing on a working deployment.

Vulnerability Details

In `postreferralswap.js`, the API makes unchecked calls to external poolparty services:

```
const resp = await fetch(
  `https://referrals.poolparty.market/api/addswapreferral?userid=${userId}&usernameemail=${userNameEmail}&usernamewallet=${userNameWallet}&site=${site}`,
  {
    credentials: 'include',
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'ngrok-skip-browser-warning': '69420',
      auth_key: POOLPARTY_REFERRAL_API_KEY,
    },
    body: JSON.stringify(verifyBody),
  },
);
```

Also, for example, in `getwaasportfolio.js` all the calls are authenticated to a 3rd party API. The application uses calls to other services globally and at many endpoints. For example, in the below case, an attacker could try to forge JWT Tokens that might trigger WAF on the external API, which will in turn block the pool party address. This is however, is not possible to prove without a working production deployment.

```
const portfolio = await fetch(
  `https://api-kl.keystonelabs.tech/v1/poolparty/getwaasportfolio`,
  {
    method: 'GET',
    headers: {
      'Authorization': `Bearer ${useraccesstoken}`,
      Accept: 'application/json',
    },
  },
);
```

```
'Content-Type': 'application/json',  
  
api_key: KL_API_KEY,  
  
site: POOLPARTY_SITE,  
  
pid: pid,  
  
walletaddress: walletAddress?.toString()  
  
},  
  
},  
  
);
```

Impact

If users make frequent requests to the PoolParty API, they could hit rate limits on the external Pool Party Market APIs before hitting any limits on the PoolParty API, resulting in unexpectedly blocking that endpoint for other users too.

Recommendation

Ensure the rate limit in the API is as strict as the external services called by the API, or whitelist the PoolParty API as a trusted caller and enforce the rate limit only on the primary Pool Party API.

Status

Resolved

[M-04] Multiple endpoints - Unchecked smart contract transaction success

Description

The application does not properly check transaction success values in smart contract interactions, potentially leading to state inconsistencies.

Vulnerability Details

In `telegramminiapp.js`, the code retrieves but does not validate the success value from transaction receipts:

```
const { receipt: { transactionHash }, success } = await wait();

// No check for 'success' value before proceeding

const sessionData = {

  // ...

  status: "ACTIVE", // Set regardless of transaction success

  // ...

};
```

This was also found in `Layout.js` - the `newUserSetup` function and `switchSmartAccountClient`.

Impact

Sessions might be recorded as active even when the underlying blockchain transactions fail, leading to inconsistent application state and a confusing user experience.

Recommendation

Always check the success value from transaction receipts and only update the state or proceed with operations if the transaction was successful. Add proper error handling for failed transactions.

Status

Resolved

Low

[L-01] API - Absence of security headers - Missing browser protections

Description

The application does not set important security headers like X-Content-Type-Options, Content-Security-Policy, and Strict-Transport-Security. Lack of these headers could allow various attacks, including clickjacking, MIME-type confusion, and protocol downgrade attacks.

Recommendation

Implement the security headers mentioned above. A reference such as <https://securityheaders.com/> can be helpful to detect compliance with best practices.

Status

Resolved

[L-02] API - Vulnerable dependencies

Description

The project was scanned for vulnerabilities in dependencies. A large number of issues in dependencies were found. This does not mean that any or all of those vulnerabilities can be exploited; however, resolving the issues decreases the likelihood of that event.

```
# npm audit report

@babel/helpers <7.26.10
Severity: moderate
Babel has inefficient RegExp complexity in generated code with .replace when
transpiling named capturing groups - https://github.com/advisories/GHSA-968p-4wvh-cqc8
fix available via `npm audit fix`
node_modules/@babel/helpers

@babel/runtime <7.26.10
```

Severity: moderate

Babel has inefficient **RegExp** complexity in generated code with **.replace** when transpiling named capturing groups - <https://github.com/advisories/GHSA-968p-4wvh-cqc8>
fix available via ``npm audit fix``
node_modules/@babel/runtime

axios 1.0.0 - 1.8.1

Severity: high

Server-Side Request Forgery in axios -

<https://github.com/advisories/GHSA-8hc4-vh64-cxmj>

axios Requests Vulnerable To Possible SSRF and Credential Leakage via Absolute URL -

<https://github.com/advisories/GHSA-jr5f-v2jv-69x6>

fix available via ``npm audit fix``

node_modules/axios

base-x <=3.0.10 || 4.0.0

Severity: high

Homograph attack allows **Unicode** lookalike characters to bypass validation. -

<https://github.com/advisories/GHSA-xq7p-g2vc-g82p>

Homograph attack allows **Unicode** lookalike characters to bypass validation. -

<https://github.com/advisories/GHSA-xq7p-g2vc-g82p>

fix available via ``npm audit fix``

node_modules/@solana/web3.js/node_modules/base-x

node_modules/base-x

node_modules/borsh/node_modules/base-x

node_modules/bs58check/node_modules/base-x

bigint-buffer *

Severity: high

bigint-buffer Vulnerable to Buffer Overflow via **toBigIntLE()** Function -

<https://github.com/advisories/GHSA-3gc7-fjrx-p6mg>

fix available via ``npm audit fix --force``

Will install @solana/spl-token@0.1.8, which is a breaking change

node_modules/bigint-buffer

@solana/buffer-layout-utils *

Depends on vulnerable versions of bigint-buffer

node_modules/@solana/buffer-layout-utils

@solana/spl-token >=0.2.0-alpha.0



Depends on vulnerable versions of @solana/buffer-layout-utils

node_modules/@solana/spl-token

@solana/web3.js 1.43.1 - 1.98.0

Depends on vulnerable versions of bigint-buffer

node_modules/@solana/web3.js

cookie <0.7.0

cookie accepts cookie name, path, and domain with out of bounds characters -

<https://github.com/advisories/GHSA-pxg6-pf52-xh8x>

fix available via `npm audit fix --force`

Will install react-cookie@8.0.1, which is a breaking change

node_modules/cookie

universal-cookie <=7.2.0

Depends on vulnerable versions of cookie

node_modules/universal-cookie

react-cookie 2.0.1 - 6.1.3

Depends on vulnerable versions of universal-cookie

node_modules/react-cookie

cross-spawn 7.0.0 - 7.0.4

Severity: high

Regular Expression Denial of Service (ReDoS) in cross-spawn -

<https://github.com/advisories/GHSA-3xgq-45jj-v275>

fix available via `npm audit fix`

node_modules/cross-spawn

elliptic <=6.6.0

Severity: critical

Elliptic's EDDSA missing signature length check -

<https://github.com/advisories/GHSA-f7q4-pwc6-w24p>

Elliptic's ECDSA missing check for whether leading bit of r and s is zero -

<https://github.com/advisories/GHSA-977x-g7h5-7qgw>

Elliptic allows BER-encoded signatures -

<https://github.com/advisories/GHSA-49q7-c7j4-3p7m>

Valid ECDSA signatures erroneously rejected in Elliptic -

<https://github.com/advisories/GHSA-fc9h-whq2-v747>

Elliptic's verify function omits uniqueness validation -

<https://github.com/advisories/GHSA-434g-2637-qmqr>



Elliptic's private key extraction in ECDSA upon signing a malformed input (e.g. a string) - <https://github.com/advisories/GHSA-vjh7-7g9h-fjfh>

fix available via `npm audit fix`

node_modules/@walletconnect/utils/node_modules/elliptic

node_modules/elliptic

@ethersproject/signing-key <=5.7.0

Depends on vulnerable versions of elliptic

node_modules/@ethersproject/signing-key

next 0.9.9 - 14.2.14

Severity: critical

Next.js missing cache-control header may lead to CDN caching empty reply -

<https://github.com/advisories/GHSA-c59h-r6p8-q9wc>

Next.js Server-Side Request Forgery in Server Actions -

<https://github.com/advisories/GHSA-fr5h-rqp8-mj6g>

Next.js Vulnerable to HTTP Request Smuggling -

<https://github.com/advisories/GHSA-77r5-gw3j-2mpf>

Next.js Denial of Service (DoS) condition -

<https://github.com/advisories/GHSA-fq54-2j52-jc42>

Denial of Service condition in Next.js image optimization -

<https://github.com/advisories/GHSA-g77x-44xx-532m>

Next.js authorization bypass vulnerability -

<https://github.com/advisories/GHSA-7gfc-8cq8-jh5f>

Next.js Allows a Denial of Service (DoS) with Server Actions -

<https://github.com/advisories/GHSA-7m27-7ghc-44w9>

Authorization Bypass in Next.js Middleware -

<https://github.com/advisories/GHSA-f82v-jwr5-mffw>

Depends on vulnerable versions of postcss

Depends on vulnerable versions of zod

fix available via `npm audit fix`

node_modules/next

node-fetch <2.6.7

Severity: high

node-fetch forwards secure headers to untrusted sites -

<https://github.com/advisories/GHSA-r683-j2x4-v87g>

No fix available

node_modules/isomorphic-fetch/node_modules/node-fetch



isomorphic-fetch 2.0.0 - 2.2.1

Depends on vulnerable versions of node-fetch

node_modules/isomorphic-fetch

fbjs 0.7.0 - 1.0.0

Depends on vulnerable versions of isomorphic-fetch

node_modules/fbjs

react 0.15.0-alpha.1 - 16.4.2

Depends on vulnerable versions of fbjs

node_modules/react-file-base64/node_modules/react

react-file-base64 *

Depends on vulnerable versions of react

node_modules/react-file-base64

postcss <8.4.31

Severity: moderate

PostCSS line return parsing error - <https://github.com/advisories/GHSA-7fh5-64p2-3v2j>

fix available via `npm audit fix`

node_modules/next/node_modules/postcss

secp256k1 4.0.0 - 4.0.3

Severity: high

secp256k1-node allows private key extraction over ECDH -

<https://github.com/advisories/GHSA-584q-6j8j-r5pm>

fix available via `npm audit fix`

node_modules/ethereumjs-util/node_modules/secp256k1

undici 6.0.0 - 6.21.0

Severity: moderate

Use of Insufficiently Random Values in undici -

<https://github.com/advisories/GHSA-c76h-2ccp-4975>

fix available via `npm audit fix --force`

Will install firebase@11.7.1, which is a breaking change

node_modules/undici

@firebase/auth 1.7.7-20240813205648 - 1.7.7-canary.1ff9661af || 1.7.8-20240827161511 - 1.7.9

Depends on vulnerable versions of undici

node_modules/@firebase/auth

firebase 0.900.22 || 7.9.1-0 - 7.9.1-canary.0396117e || 8.10.0-20217172214 -



```

8.10.0-canary.f40c0db53 || 9.0.0-20217250818 - 9.0.0-canary.d0d3acb10 ||
9.0.1-2021727231341 - 9.0.1-canary.e039e1472 || 9.0.2-2021891633 -
9.0.2-canary.ff9baf70c || 9.0.3-202181503543 - 9.1.0-canary.f7d8324a1 ||
9.1.1-2021830195733 - 9.1.1-canary.e70de6201 || 9.1.2-20219523556 -
9.1.2-canary.fc1d36497 || 9.22.1-20230524195328 || 10.12.2-20240523185724 -
10.12.2-canary.872e6f63e || 10.13.0-20240813205648 - 10.13.0-canary.fa0ed08fb ||
10.13.1-20240827161511 - 10.14.1
  Depends on vulnerable versions of @firebase/auth
  Depends on vulnerable versions of @firebase/auth-compat
  Depends on vulnerable versions of @firebase/firestore
  Depends on vulnerable versions of @firebase/firestore-compat
  Depends on vulnerable versions of @firebase/functions
  Depends on vulnerable versions of @firebase/functions-compat
  Depends on vulnerable versions of @firebase/storage
  Depends on vulnerable versions of @firebase/storage-compat
node_modules/firebase
@firebase/auth-compat 0.5.12-20240813205648 - 0.5.12-canary.fa0ed08fb ||
0.5.13-20240827161511 - 0.5.14
  Depends on vulnerable versions of @firebase/auth
  Depends on vulnerable versions of undici
node_modules/@firebase/auth-compat
@firebase/firestore 4.7.1-20240827161511 - 4.7.3
  Depends on vulnerable versions of undici
node_modules/@firebase/firestore
  @firebase/firestore-compat <=0.0.900-exp.520ca39d0 || 0.3.35-20240813205648 -
0.3.35-canary.fa0ed08fb || 0.3.36-20240827161511 - 0.3.38
  Depends on vulnerable versions of @firebase/firestore
node_modules/@firebase/firestore-compat
@firebase/functions 0.11.7-20240827161511 - 0.11.8
  Depends on vulnerable versions of undici
node_modules/@firebase/functions
  @firebase/functions-compat 0.3.13-20240827161511 - 0.3.14
  Depends on vulnerable versions of @firebase/functions
node_modules/@firebase/functions-compat
@firebase/storage 0.13.1-20240827161511 - 0.13.2
  Depends on vulnerable versions of undici
node_modules/@firebase/storage
  @firebase/storage-compat <=0.0.900-exp.520ca39d0 || 0.1.4-202192711727 -

```

```
0.1.4-canary.f27fe4304 || 0.3.10-20240814182916 - 0.3.10-canary.fa0ed08fb ||
0.3.11-20240827161511 - 0.3.12
```

Depends on vulnerable versions of @firebase/storage
node_modules/@firebase/storage-compat

```
ws 7.0.0 - 7.5.9
```

Severity: high

ws affected by a DoS when handling a request with many HTTP headers -

<https://github.com/advisories/GHSA-3h5v-q93c-6h6q>

fix available via `npm audit fix`

```
node_modules/@ethersproject/providers/node_modules/ws
```

```
@ethersproject/providers <=5.7.2
```

Depends on vulnerable versions of ws

```
node_modules/@ethersproject/providers
```

```
ethers 5.0.15 - 5.7.2
```

Depends on vulnerable versions of @ethersproject/providers

Depends on vulnerable versions of @ethersproject/signing-key

```
node_modules/ethers
```

```
zod <=3.22.2
```

Severity: moderate

Zod denial of service vulnerability - <https://github.com/advisories/GHSA-m95q-7qp3-xv42>

fix available via `npm audit fix`

```
node_modules/next/node_modules/zod
```

36 vulnerabilities (4 low, 14 moderate, 16 high, 2 critical)

To address issues that do not require attention, run:

```
npm audit fix
```

To address all issues possible (including breaking changes), run:

```
npm audit fix --force
```

Some issues need review, and may require choosing a different dependency.

Recommendation

Upgrade the packages to secure versions where possible. On the other hand, pay attention to whether the upgrade does not break business logic.

Status

Resolved

[L-03] Multiple endpoints - Hardcoded authorization bypass due to placeholders in code

Description

Multiple API endpoints bypass authentication by using hardcoded values instead of verifying the provided authorization token, allowing any user to access sensitive API endpoints without credentials.

Vulnerability Details

In multiple API files (`getuser.js`, `getwaasportfolio.js`, `getsinglealphacall.js`, `gettokenlists.js`):

```
const auth = 'success'; // params.authorization;

// console.log('params', auth);

// Authentication is bypassed due to a hardcoded 'success' value

if (!params) {

  return res.status(401).json({

    error: 'No params information',

  });

} else if (!auth && params) {

  return res.status(401).json({

    error: 'Not authorized',

  });

}
```

```
} else if (auth && params) {  
  
  // But the token is still extracted and forwarded  
  
  let useraccesstoken = params.authorization.replace('Bearer ', '');  
  
  // And sent to an external API  
  
  const portfolio = await fetch(  
  
    `https://api-kl.keystonelabs.tech/v1/poolparty/getwaasportfolio`,  
  
    {  
  
      method: 'GET',  
  
      headers: {  
  
        'Authorization': `Bearer ${useraccesstoken}`,  
  
        Accept: 'application/json',  
  
        'Content-Type': 'application/json',  
  
        api_key: KL_API_KEY,  
  
        // ...  
  
      },  
  
    },  
  
  );  
  
}
```

Moreover, this possibly could allow slipping additional payloads to a 3rd party api, which could have exceptions to the PoolParty API IP addresses. However, without a working production deployment, it is difficult to prove the exact impact.

Impact

Anyone can access the API without valid credentials due to the hardcoded auth check. Possibly the requests can be modified in an unspecified way - requires testing on ready deployment.

Recommendation

Remove hardcoded authentication values, properly validate tokens before use, and only forward valid tokens to external services.

Status

Resolved



QA

[Q-01] Multiple occurrences - `console.log` in production React code

Description

Production React components contain numerous `console.log` statements, which are typical for debug/dev environments, but not needed for users. Throughout the codebase, particularly in `Layout.js`:

```
console.log('Handling auth state changes...', isConnected, isReconnecting)
console.log('Setting up smart wallet')
console.log('Setting OTC CA', contract, selectedChain)
console.log(`Switching chain: ${prevChainId} -> ${state.activeChainId}`)
```

Recommendation

Remove debug `console.log` statements or implement conditional logging for development environments only.

Status

Acknowledged

[Q-02] Multiple occurrences - Dead code in API routes - unreachable conditional blocks

Description

Several API routes contain conditional logic that can never be executed, specifically in the hardcoded auth part in `getwaasportfolio.js` and other files implementing this pattern.

There is a check if the params are present, but the params are HTTP headers, which are part of every HTTP request. Therefore, it is not possible that they will not be present. The first condition, `if (!params)`, is effectively dead code because Next.js API routes always have a `req.headers` object (even if empty), so params will never be undefined or null. Additionally, since `auth` is hardcoded to `'success'`, the second condition (`!auth && params`) can never be true. While not directly a security risk, dead code makes maintenance more difficult.

Recommendation

Remove or refactor unreachable code blocks, and ensure that conditional statements reflect actual logic that can be executed.

Status

Resolved

Conclusion

After Hashlock's analysis, the PoolParty project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

